

mTCP: A User-level TCP Stack



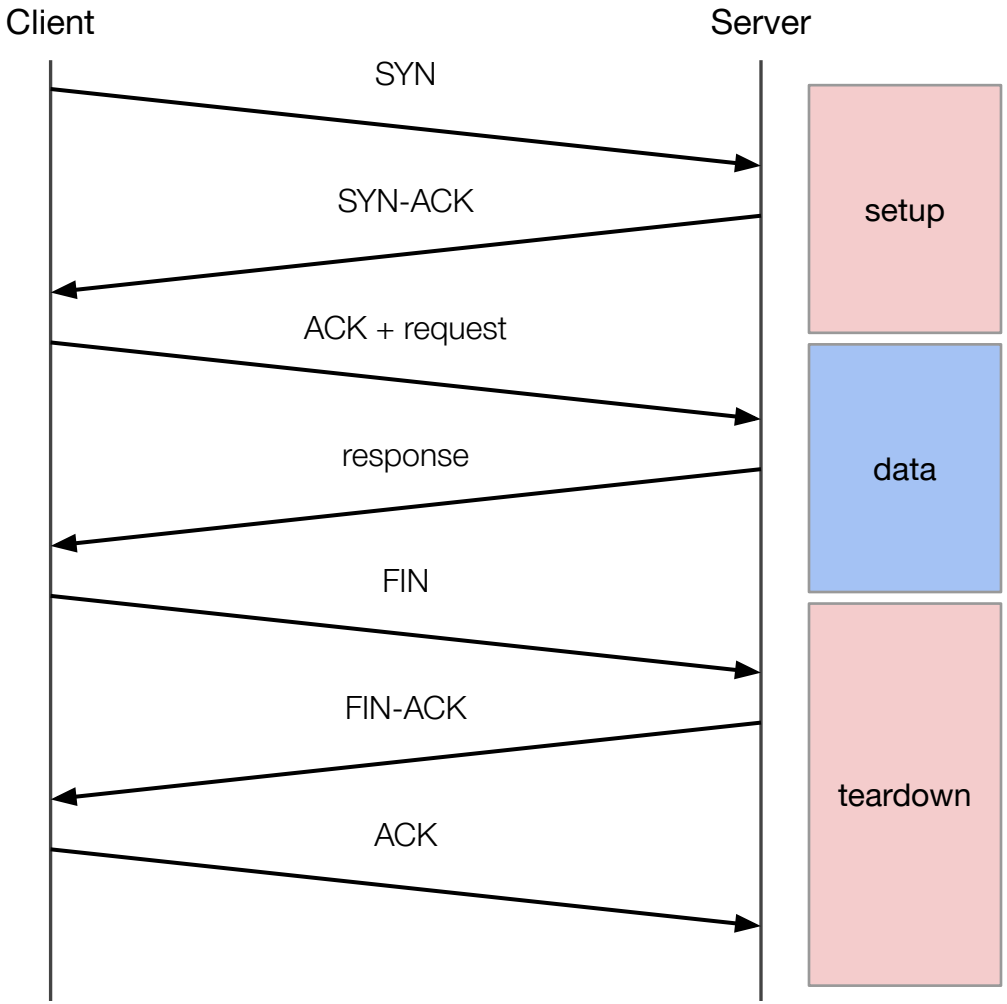
Mainack Mondal
Sandip Chakraborty

CS 60203
Autumn 2026

Outline

- Why short TCP connections are hard for the kernel
- Bottlenecks in the Linux TCP stack, and earlier fixes
- The base idea of mTCP: user level, per core, batched
- Architecture: app thread, mTCP thread, packet I/O
- Key concepts: per-core partitioning, batching, the mTCP loop
- Programming model and results
- mTCP vs Linux TCP vs XDP
- Applications, limitations and summary

Motivation: Short Connections Everywhere



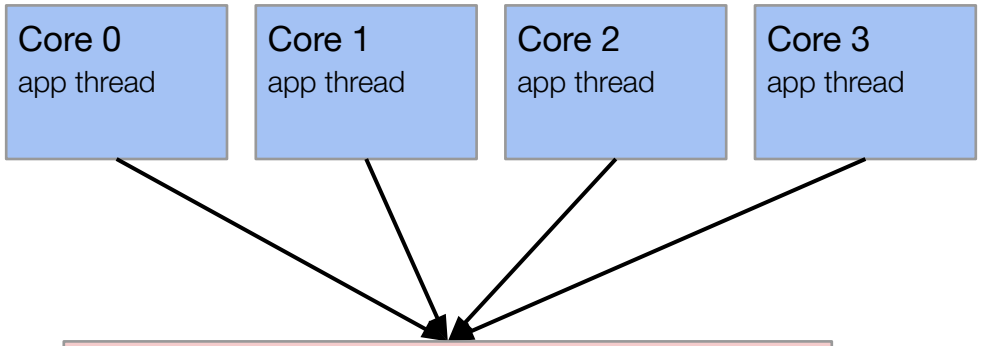
Short flows dominate many workloads

- Web requests, REST and RPC calls, key-value lookups, mobile apps
- Most connections carry only a few packets of data

So the per-connection cost matters most

- accept, socket creation, file descriptor, close: paid for every connection
- The key metric is **connections per second**
- It should grow linearly with the number of cores; on Linux it does not

Why the Linux TCP Stack Does Not Scale



One listen socket, one accept queue
protected by one lock

One file descriptor table per process
"lowest free number" rule forces sharing

Every core competes for the same queue and table.

Shared resources

- Accept queue and its lock
- File descriptor table

Broken locality

- Kernel handles a packet on one core, the app may run on another
- Cache misses and bouncing cache lines

System call overhead

- Every accept, read, write, close enters the kernel
- Small messages pay it again and again

Heavy sockets

- Each socket is a VFS file (inode, dentry)
- Costly to create and destroy per connection

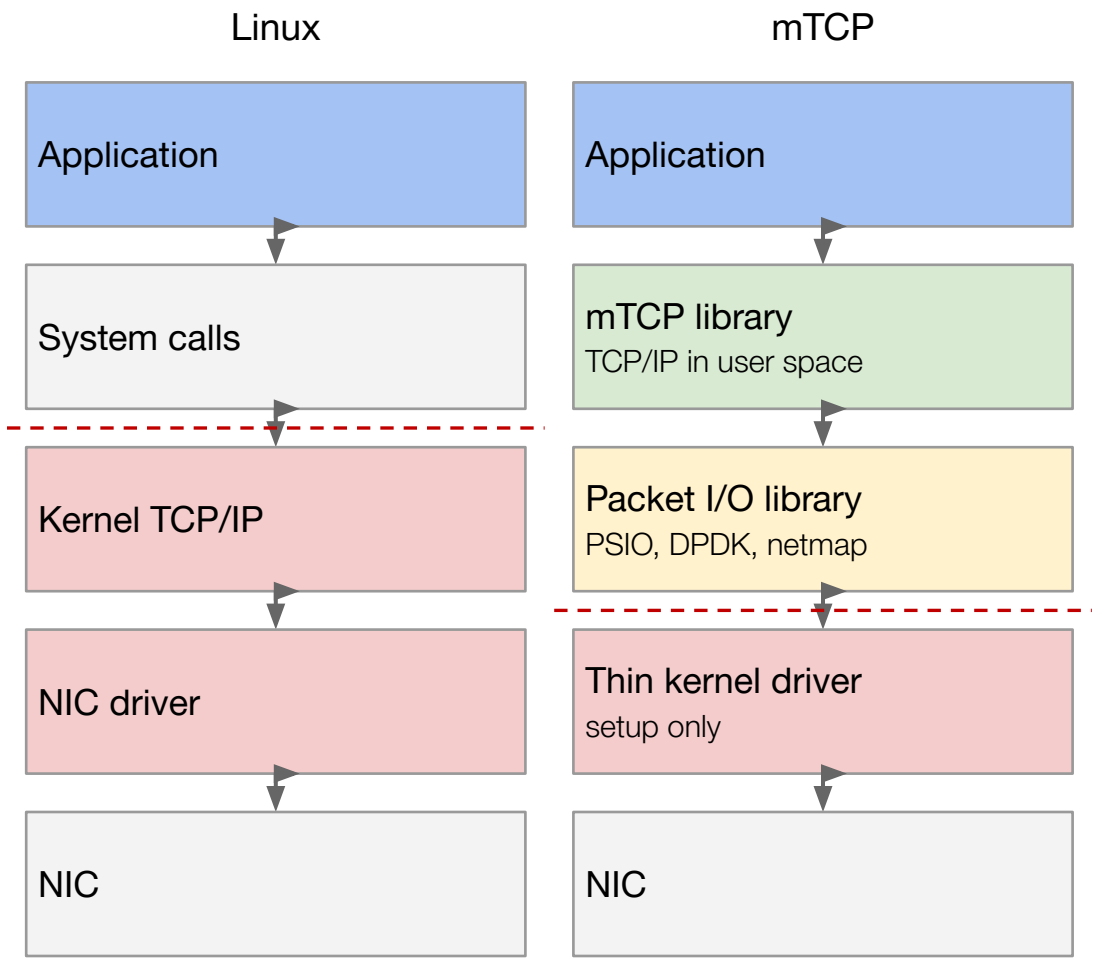
Result: for short connections, most CPU time goes to kernel overhead, not to the application.

Earlier Fixes Were Partial

System	Accept queue	Connection locality	System call batching	Packet I/O batching	API
Linux (3.x)	Shared	No	No	No	BSD sockets
Linux + SO_REUSEPORT (3.9+)	Per core	Not guaranteed	No	No	BSD sockets
Affinity-Accept (EuroSys 2012)	Per core	Yes	No	No	BSD sockets
MegaPipe (OSDI 2012)	Per core	Yes	Yes	No	New lightweight socket API
mTCP (NSDI 2014)	Per core	Yes	No system calls on the fast path	Yes	BSD-like, at user level

All earlier systems keep the kernel TCP stack, so they still pay kernel costs for every packet. mTCP removes the kernel from the fast path.

The Base Idea of mTCP



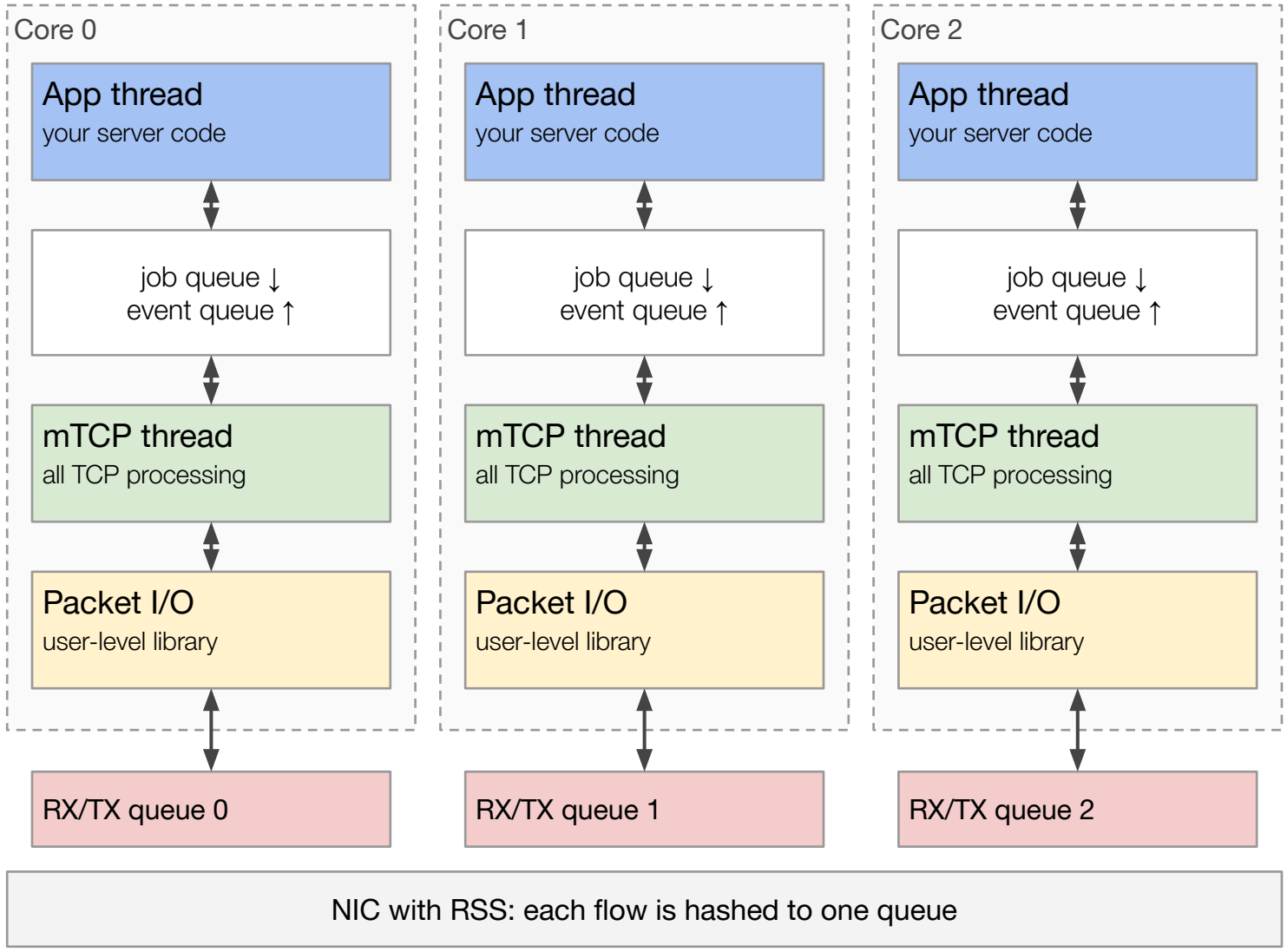
red dashed line: user/kernel boundary

1. Move TCP to user space
   - Built on a fast user-level packet I/O library
   - No system call, no VFS, no kernel socket per operation

2. Partition everything per core
   - Each core has its own connections, queues and tables
   - No sharing between cores, so no locks

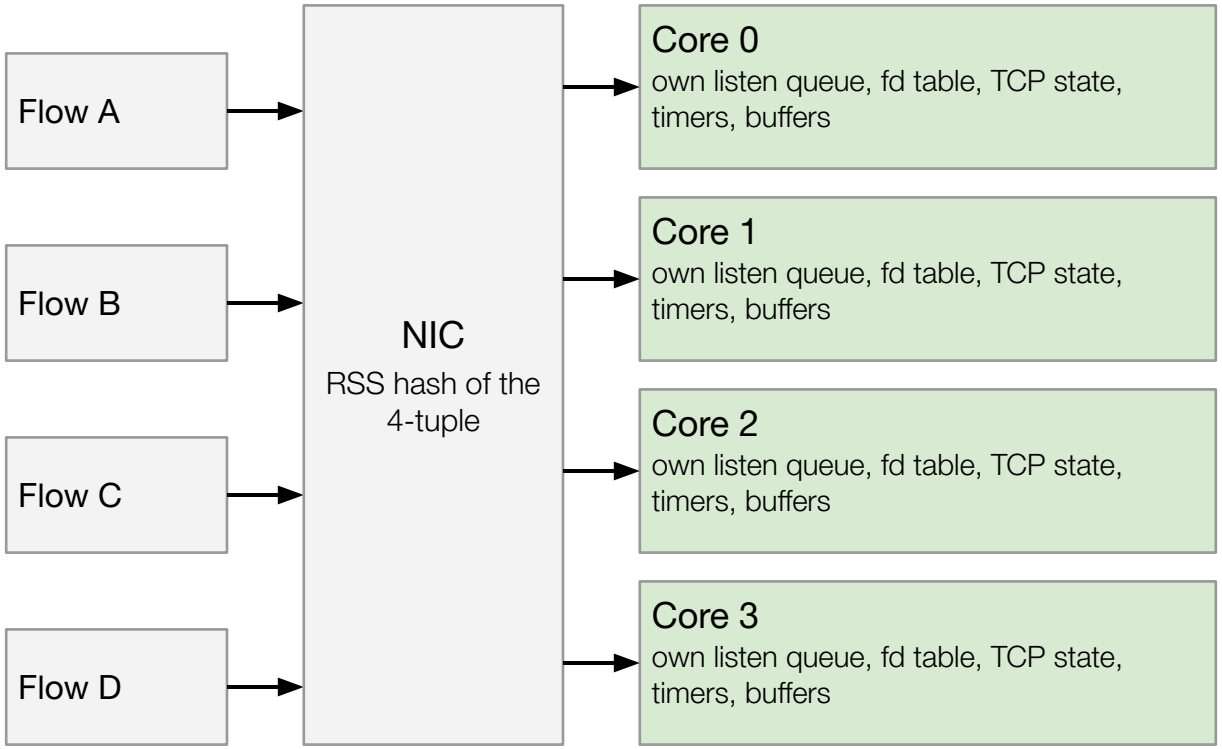
3. Batch everything
   - Packets, events and app requests are handled in groups
   - Fixed costs are paid once per batch, not once per item

mTCP Architecture



- One app thread and one mTCP thread per core, pinned to the same core
- They talk only through per-core queues: **jobs go down** (connect, send, close), **events come up** (new connection, data ready)
- The mTCP thread does all TCP work and drives its own NIC queue
- No data is shared across cores
- Packet I/O: PSIO (PacketShader I/O) in the paper; DPDK and netmap later

Key Concept 1: Per-Core Partitioning

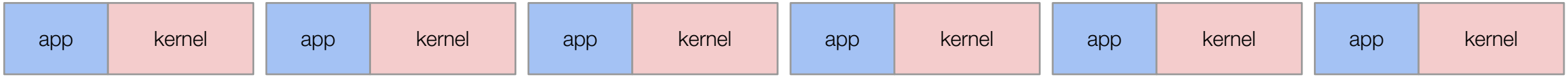


All packets of one connection always reach the same core.

- **Incoming connections:** RSS keeps every packet of a flow on one queue, and so on one core
- **Outgoing connections:** mTCP picks the source port so that the replies hash back to the same core
- Every core keeps its own listen queue, file descriptor table, connection table, timers and memory pools
- App thread on the same core: the data stays in that core's cache
- Result: no locks on the fast path and close to linear scaling with cores

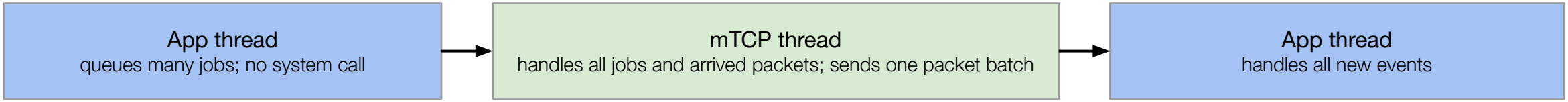
Key Concept 2: Batching at Every Level

Linux: one system call (user → kernel → user) per operation



each switch costs a mode change, and pollutes caches and TLBs

mTCP: work is queued, then handled in one go



Packet batching

- Receive and send many packets per call to the packet I/O library
- Fewer interrupts and register writes

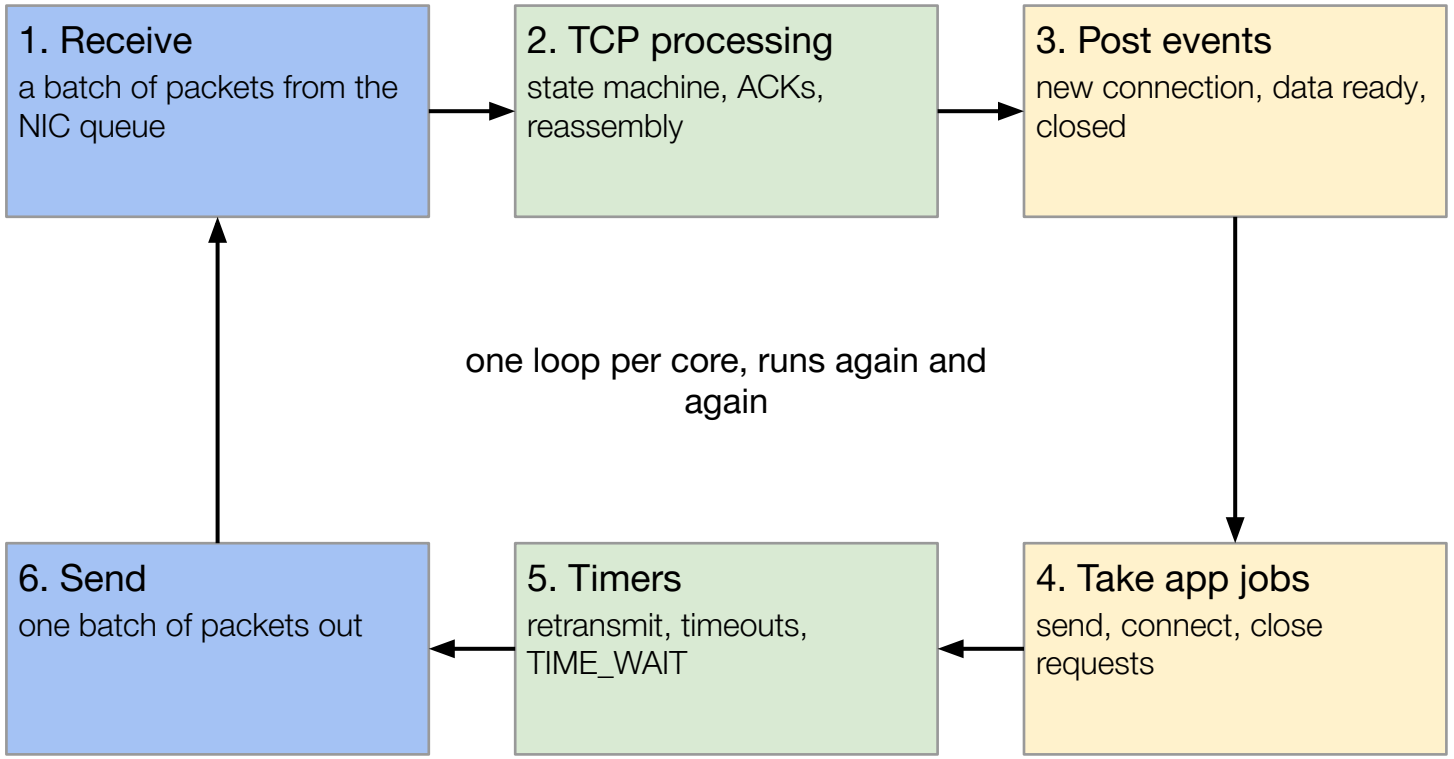
Job and event batching

- App calls only put jobs in a queue
- mTCP posts many events at once; one thread switch serves the whole group

The trade-off

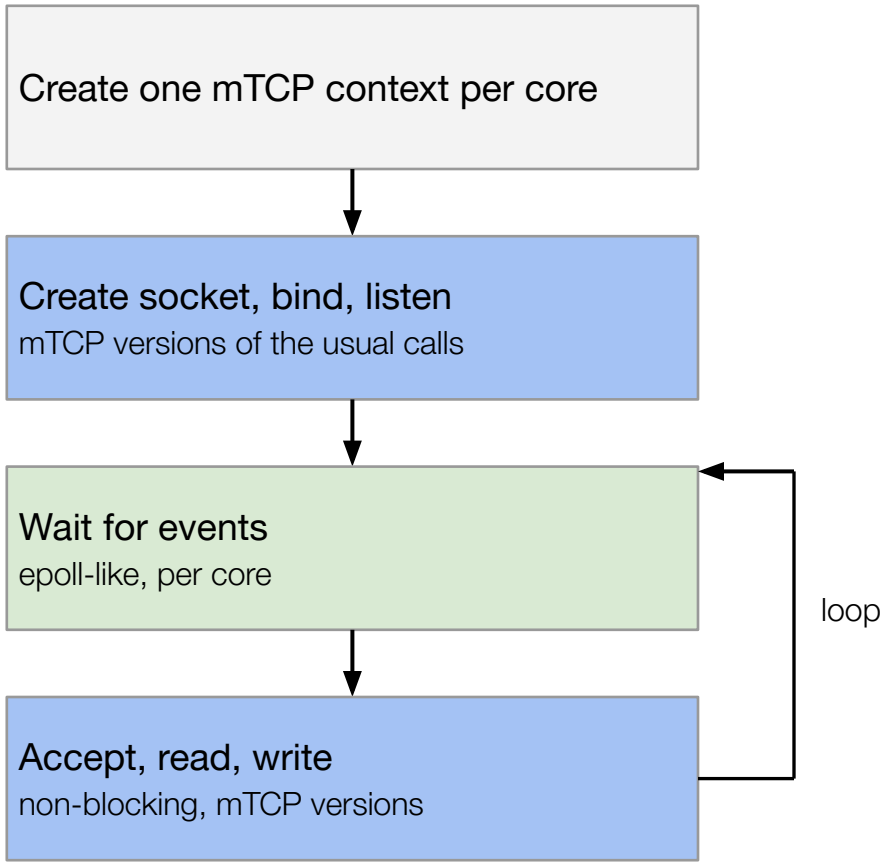
- Waiting to fill a batch can add a little latency
- Pays off at high load, where throughput matters most

Key Concept 3: The mTCP Thread Loop



- Outgoing packets are sent in priority order: **control** (SYN, FIN, RST) first, then **pure ACKs**, then **data**
- Timers live in per-core lists, so no global timer lock
- The app thread is woken only when events are ready
- Polling plus batching keep the cost per packet small

Programming Model



Same shape as an epoll-based server on Linux.

Familiar API

- BSD-like socket calls and epoll-like events
- Porting is mostly renaming calls and creating per-core contexts

Rules to follow

- One thread per core, event-driven, non-blocking
- Fork-per-connection or blocking designs do not fit

Ported in the paper

- lighttpd web server, ApacheBench load generator, SSLShader SSL proxy
- Porting mainly changed the event loop and socket calls

Results from the Paper (NSDI 2014)

25x

faster than Linux for small message transactions on short connections

3x

faster than MegaPipe, the best earlier research system

33 to 320%

faster real applications (SSLShader to lighttpd) than on Linux

Where the gain comes from

- No system calls, no VFS, no kernel socket objects
- Batching of packets, jobs and events
- Per-core data: no locks, warm caches, near-linear scaling

Where the gain is small

- Long bulk transfers: Linux is already efficient there
- Light load: batches stay small, and latency matters more

mTCP vs Linux TCP vs XDP

Aspect	Linux TCP stack	XDP	mTCP
What it is	Full TCP/IP in the kernel	Packet hook in the NIC driver	Full TCP/IP in user space
Handles TCP connections?	Yes	No: per-packet decisions only	Yes
Kernel work per packet	Full stack	Driver only	None on the fast path
Sharing the NIC	All applications	Yes; other packets go to the stack	Owned by one application
CPU use when idle	Low	Low	Polling cores stay busy
Safety	Trusted kernel code	Verifier-checked program	App and stack in one process
Best for	General use	Filters, load balancers, forwarding	Servers with many short connections

They solve different problems and can be combined: AF_XDP can serve as the packet I/O layer under a user-level stack.

Applications and Legacy

Web servers and proxies

- High connection-rate HTTP servers such as lighttpd
- SSL front ends (SSLShader)

Load generators

- Generate millions of short connections from a few machines
- Testing servers and middleboxes

Middleboxes

- mOS (NSDI 2017) builds a stateful middlebox stack on mTCP
- Flow tracking at TCP level

Runs on modern I/O

- DPDK and netmap support added after the paper
- Open source on GitHub

Influence on industry

- User-space stacks such as F-Stack and Seastar use similar ideas
- Per-core, share-nothing, batched

Influence on research

- IX, TAS and others: dataplane OS designs for fast TCP
- Same goal with stronger protection

Limitations of mTCP

Design trade-offs

- The NIC (or its queues) belongs to one application; kernel tools like tcpdump and iptables do not see the traffic
- Polling threads keep cores busy even when idle
- Batching adds some latency at low load
- No kernel protection between the app and its TCP stack

Practical issues

- TCP features are a subset of Linux (options, congestion control choices, offloads)
- Applications must be ported and made event-driven
- Needs a supported NIC and packet I/O library
- Less mature and less tested than the kernel stack

Lesson that lasted: share nothing across cores, batch fixed costs, and keep the kernel off the fast path. Later systems kept these ideas and added better protection.

Summary

- Short TCP connections stress the per-connection path; Linux connection rate does not scale with cores
- Causes: shared accept queue and fd table, broken cache locality, system call and VFS overheads
- mTCP moves TCP to user space on top of fast packet I/O
- Per-core partitioning with RSS and port choice: no shared state, no locks
- Batching of packets, jobs and events amortises fixed costs
- BSD-like and epoll-like API makes porting event-driven servers easy
- Large gains for short connections; trade-offs in isolation, CPU use and features

References

- E. Jeong et al., mTCP: a Highly Scalable User-level TCP Stack for Multicore Systems, NSDI 2014
- A. Pesterev et al., Improving Network Connection Locality on Multicore Systems (Affinity-Accept), EuroSys 2012
- S. Han et al., MegaPipe: A New Programming Interface for Scalable Network I/O, OSDI 2012
- M. Jamshed et al., mOS: A Reusable Networking Stack for Flow Monitoring Middleboxes, NSDI 2017
- A. Belay et al., IX: A Protected Dataplane Operating System for High Throughput and Low Latency, OSDI 2014
- mTCP source code: github.com/mtcp-stack/mtcp